STW7071CEM INFORMATION RETRIEVAL

SUBMITTED TO: SIDDHARTHA NEUPANE

Submitted by: Albert Maharjan
10248931

June 3, 2023

Introduction

This project aims to develop a vertical search engine similar to Google Scholar, but with a specific focus on papers and books published by members of Coventry University (CU). The system will crawl "https://pureportal.coventry.ac.uk/en/persons/" profiles of CU academic employees and index their articles in their profiles to create an inverted index. The system will periodically check for new information, ideally as a scheduled job, to keep the index up-to-date.

From the user's perspective, the search engine will provide an interface similar to Google Scholar's main page. Users can enter their queries or keywords related to the resources they are looking for. The system will display the search results sorted by relevance, following a similar approach to Google Scholar. However, the search engine will exclusively retrieve articles from CU with at least one co-author affiliated with the university.

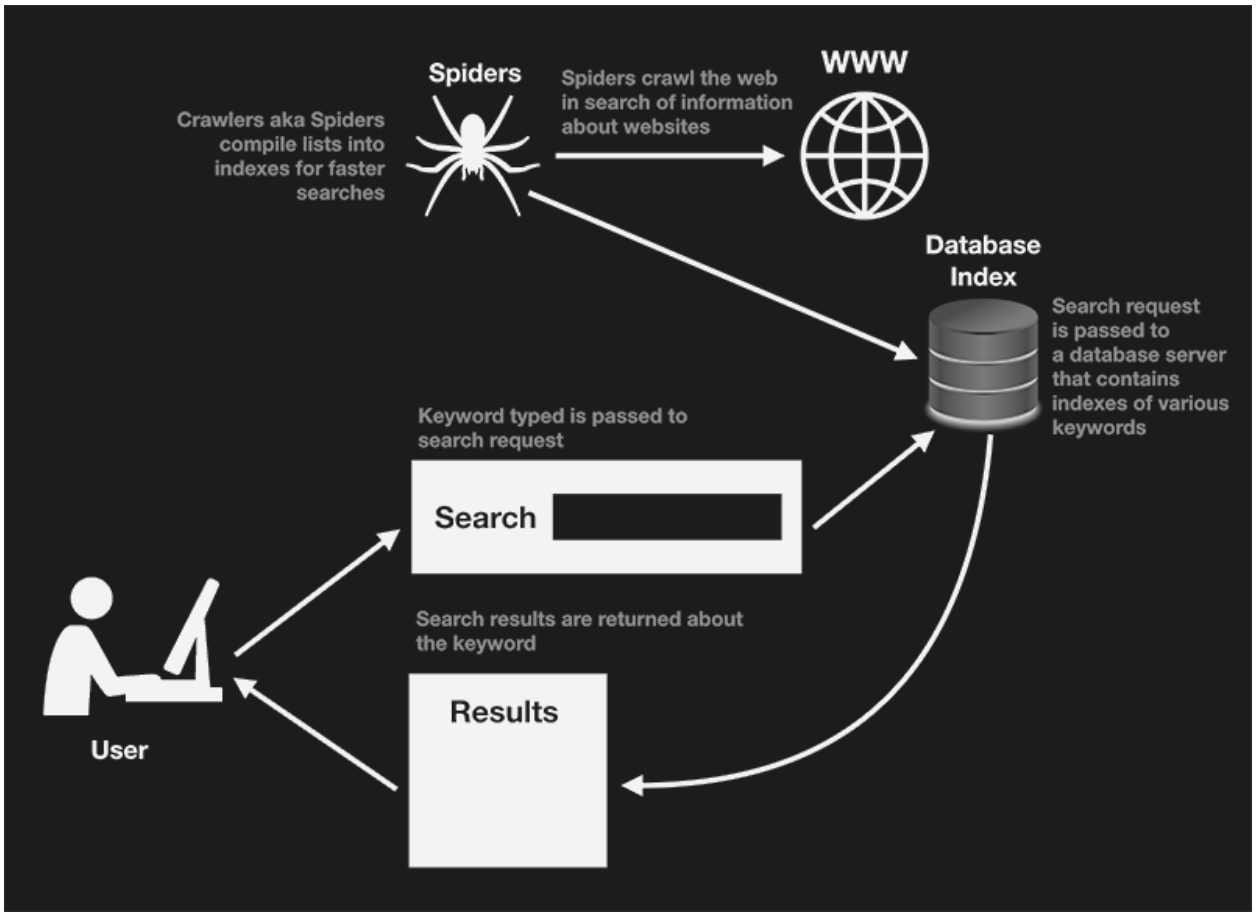


Fig 1: Crawler overview

In addition to the search functionality, the system will also include a subject classification feature. This feature will take a scientific document as input and classify it into one or more categories such as 'Computer Science', 'Medicine and Dentistry', 'Veterinary Science and Veterinary Medicine' or

‘Engineering', which are all areas of study. The subject classification functionality can be developed as a standalone software or integrated with the search engine.

Crawler

A crawler is a software application that performs automated searches of web documents. Its main purpose is to navigate the internet and create an index, making it particularly useful for search engines. Crawlers are designed to perform repetitive tasks, automating the process of browsing.

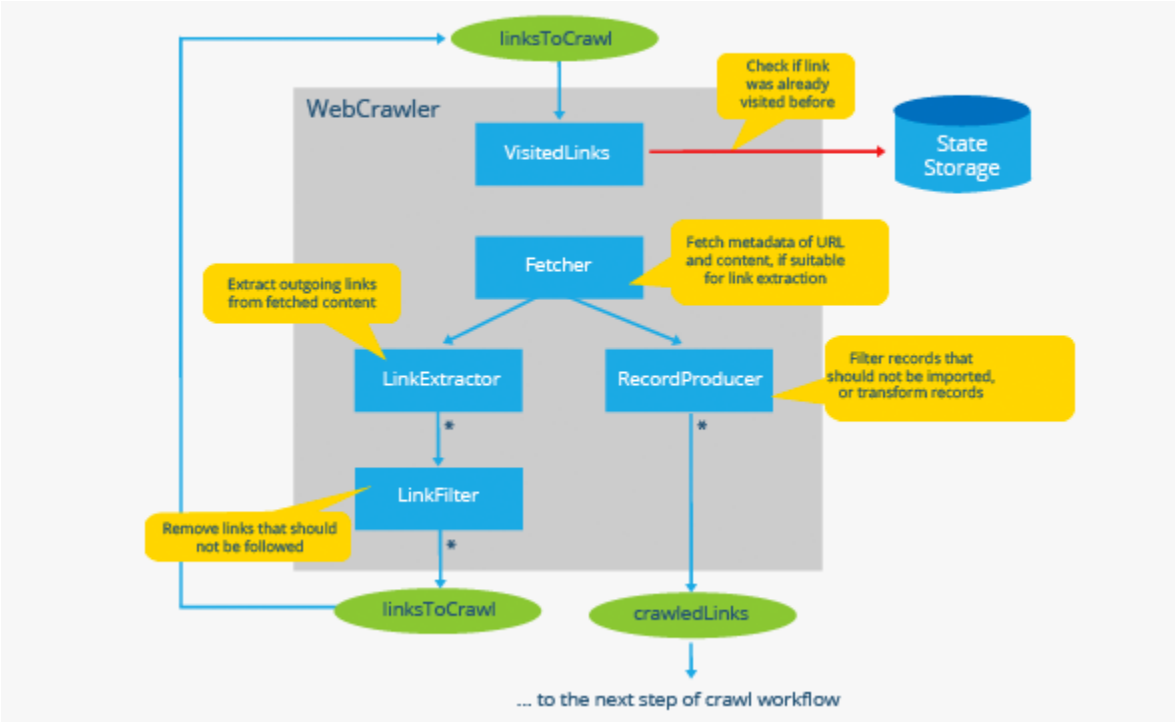


Fig 2: Crawler architecture

Crawlers visit websites and read their pages and other information to create entries for a search engine's index. The primary purpose of a web crawler is to provide users with a comprehensive and up-to-date index of all available online content. By using web crawlers, businesses can keep their online presence (i.e. SEO, frontend optimization, and web marketing) up-to-date and effective. Search engines like Google, Bing, and Yahoo use crawlers to properly index downloaded pages so that users can find them faster and more efficiently when searching. Without web crawlers, there would be nothing to tell them that your website has new and fresh content.

Find the authors

```
nextLink = driver.find_element(By.CSS_SELECTOR,".nextLink").is_enabled()
while (nextLink):
    page = driver.page_source
    bs = BeautifulSoup(page, "lxml")

    for link in bs.findAll('a', class_='link person'):
        url = str(link)[str(link).find('https://pureportal.coventry.ac.uk/en/persons/'):].split('"')
        Links.append(url[0])
    try:
        if driver.find_element(By.CSS_SELECTOR,".nextLink"):
            element = driver.find_element(By.CSS_SELECTOR,".nextLink")
            driver.execute_script("arguments[0].click();", element)
        else:
            nextLink = False
    except NoSuchElementException:
        break
write_authors(Links, 'Authors_URL.txt')
```

Fig 3: Finding arthors

Firstly store all the author's links to a text file to crawl in the latter phase. This function checks if there is a next button, and will iterate to store the link for "a" tag from the browser until there is no next button.

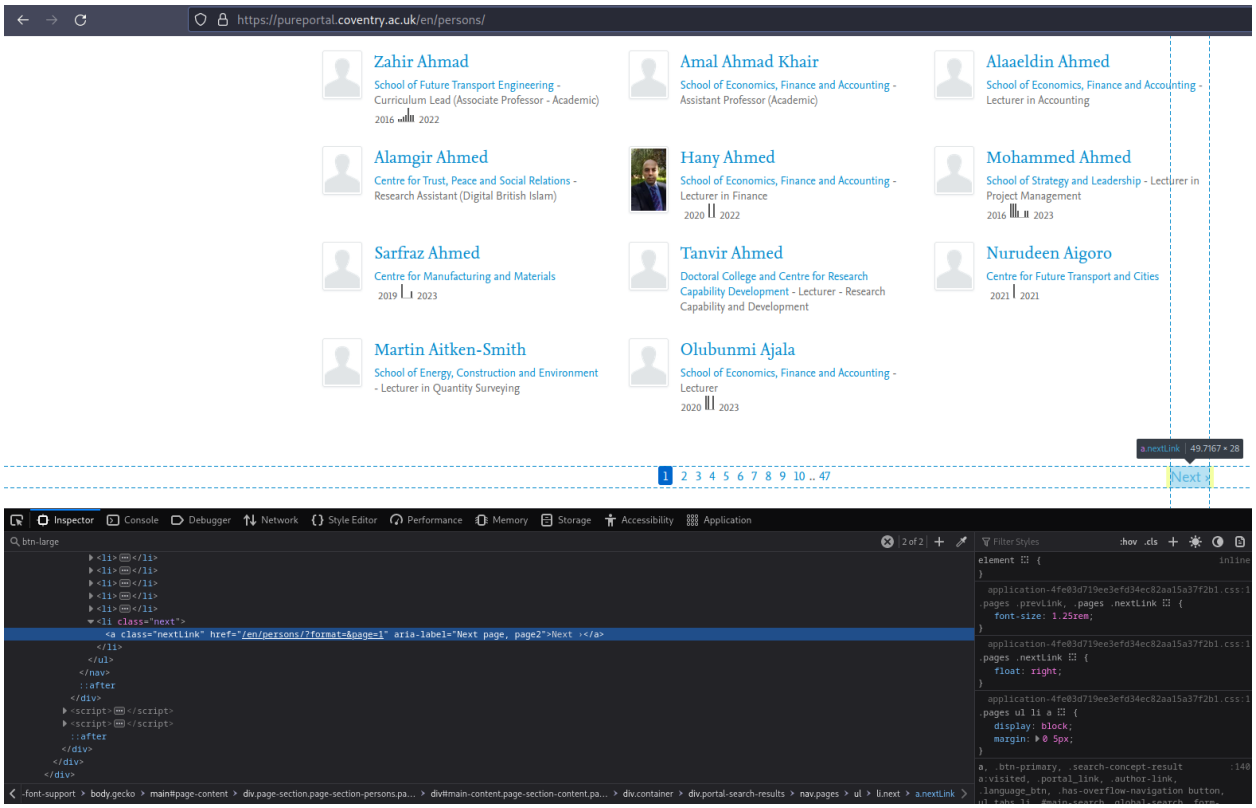


Fig 4: Crawler finding next pages

```
if driver.find_elements(By.CSS_SELECTOR,".portal_link.btn-primary.btn-large"):
    element = driver.find_elements(By.CSS_SELECTOR,".portal_link.btn-primary.btn-large")
    for a in element:
        if "research output".lower() in a.text.lower():
            driver.execute_script("arguments[0].click();", a)
            driver.get(driver.current_url)
            r = requests.get(driver.current_url)
            soup = BeautifulSoup(r.content, 'lxml')
            table = soup.find('ul', attrs={'class': 'list-results'})
            if table != None:
                for row in table.findAll('div', attrs={'class': 'result-container'}):
                    data = {}
                    data['name'] = row.h3.a.text
                    data['pub_url'] = row.h3.a['href']
                    date = row.find("span", class_="date")

                    rowitem = row.find_all(['div'])
                    span = row.find_all(['span'])
                    data['cu_author'] = name
                    data['date'] = date.text

                    l3 = crawlDetail(driver, data['pub_url'])
                    data["abstract"] = l3[0]
                    data["category"] = l3[1]
                    pub_data.append(data)
```

Fig 5: Crawler source code to find more publications

Next step is to go through all the links of the author to retrieve the data of the publications. There are two cases to get publication. The website shows “View all x research outputs” if the author has more than 5 publications. If the case is true, it clicks to expand and get all the data of publication.

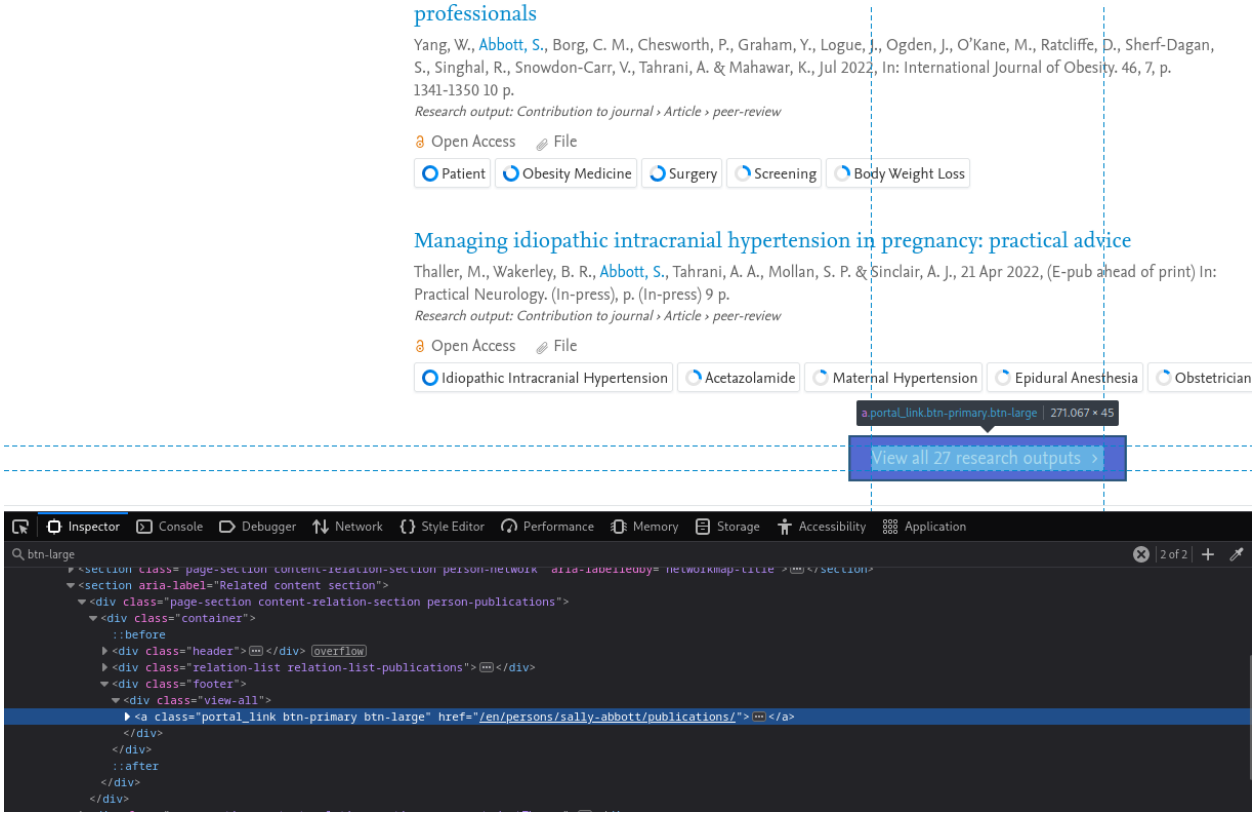


Fig 6: Crawler to find more publications

```
    else:
        r = requests.get(link)
        soup = BeautifulSoup(r.content, 'lxml')
        table = soup.find('div', attrs={'class': 'relation-list relation-list-publications'})
        if table != None:
            for row in table.findAll('div', attrs={'class': 'result-container'}):
                data = {}
                data["name"] = row.h3.a.text
                data['pub_url'] = row.h3.a['href']
                date = row.find("span", class_="date")
                rowitem = row.find_all(['div'])
                span = row.find_all(['span'])
                data['cu_author'] = name
                data['date'] = date.text

                l3 = crawlDetail(driver, data['pub_url'])
                data["abstract"] = l3[0]
                data["category"] = l3[1]
                pub_data.append(data)
```

Fig 7: Crawler to find general publications

Fig 8: Crawler's behavior to finding html compoment

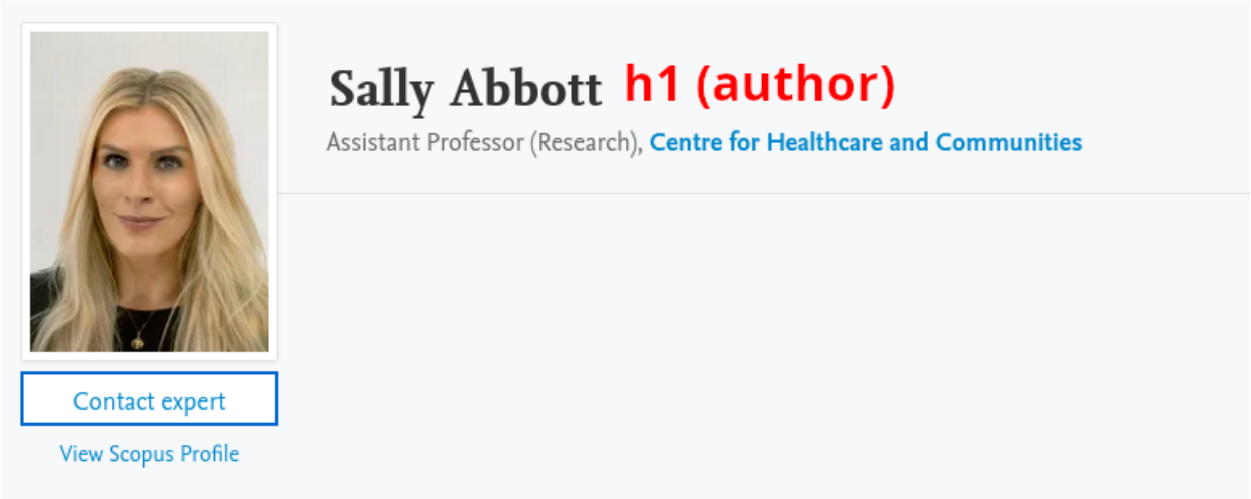


Fig 9: Crawler's behavior to finding html compoment 2

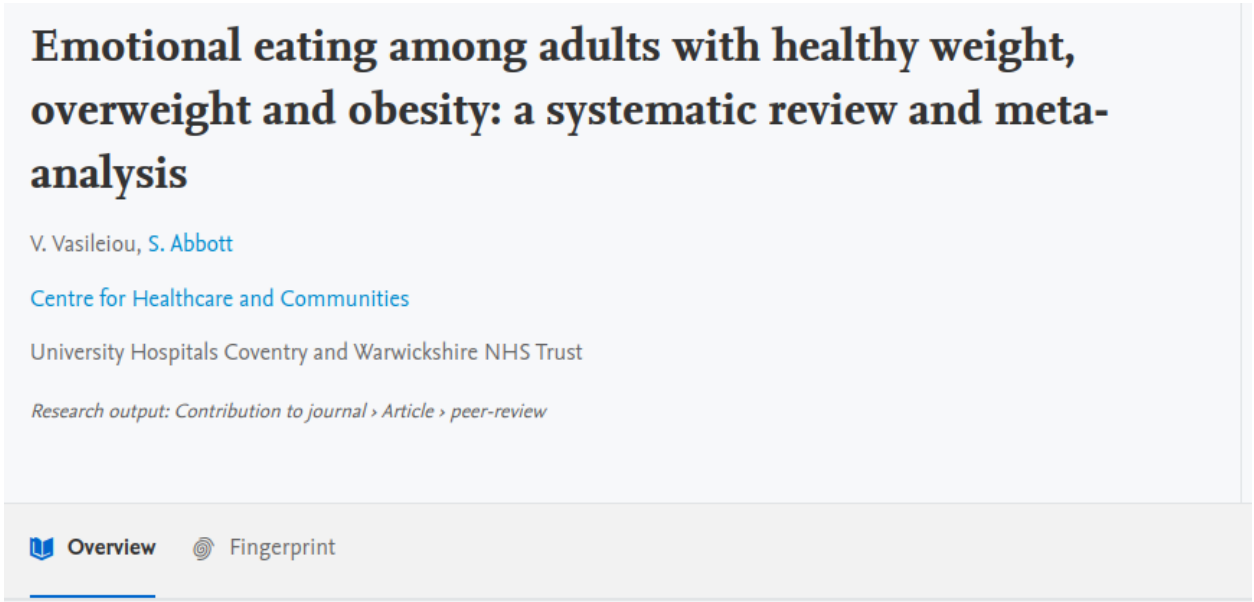
Find the publications

The publication is further crawled to get extra information such as abstract and the fingerprints of an individual publication.

```python
def crawlDetail(driver, uri):
    driver.get(uri)
    abstract = driver.find_element(
        By.CSS_SELECTOR,"div[class='content-content publication-content']>div"
    )
    abs_text= abstract.text if abstract is not None else ""
    uri = uri + "/fingerprints/"
    driver.get(uri)
    finger_print = driver.find_elements(
        By.CSS_SELECTOR,"div[class='publication-fingerprint-thesauri']>h3"
    )
    finger_text = [x.text for x in finger_print] if finger_print is not None else []
    return [
        abs_text,
        finger_text
    ]
```

Fig 10: Crawler source code to find abstract and fingerprints

The content of Abstract from the following figure is extracted for further data collection.



Emotional eating among adults with healthy weight, overweight and obesity: a systematic review and meta-analysis

V. Vasileiou, S. Abbott

Centre for Healthcare and Communities

University Hospitals Coventry and Warwickshire NHS Trust

Research output: Contribution to journal › Article › peer-review

Overview Fingerprint

Abstract

Background: Emotional eating (EE) is a disordered eating behaviour which may lead to overeating. It is not clear whether EE presents to an equal degree among adults, regardless of their body mass index (BMI) status. The aim of this study was to assess whether there is a difference in degree of EE between adults with healthy weight, overweight and obesity. Methods: MEDLINE and APA PsycINFO databases were searched from inception up to January 2022 for studies that reported EE scores from validated questionnaires. The quality of all included studies was assessed using the AXIS tool. Meta-analysis used random effects and standardised mean difference (SMD). Heterogeneity was investigated usingI2statistics and sensitivity analyses. Results: A total of 11 studies with 7207 participants were included in the meta-analysis. Degree of EE was greater in adults with a BMI above the healthy range, compared to adults with a healthy BMI (SMD 0.31, 95% CI 0.17 to0.45;I2= 85%). However, subgroup analysis found that degree of EE was greater only in adults with obesity (SMD 0.61, 95% CI 0.41 to 0.81;I2= 62%),and there was no difference in degree of EE between adults with overweight and those with a healthy BMI. Conclusions: Degree of EE is greater among adults living with obesity, compared to adults with a healthy BMI, indicating a need for behavioural support to support EE among people living with obesity seeking weight management. Future research should examine the long-term effectiveness of interventions for EE

Fig 11: Crawler finds abstract from a publication

The final step is to get the categorical data which is accessed by redirecting to the fingerprint section. From the following figure, the values “Neuroscience” and “Psychology” are extracted.

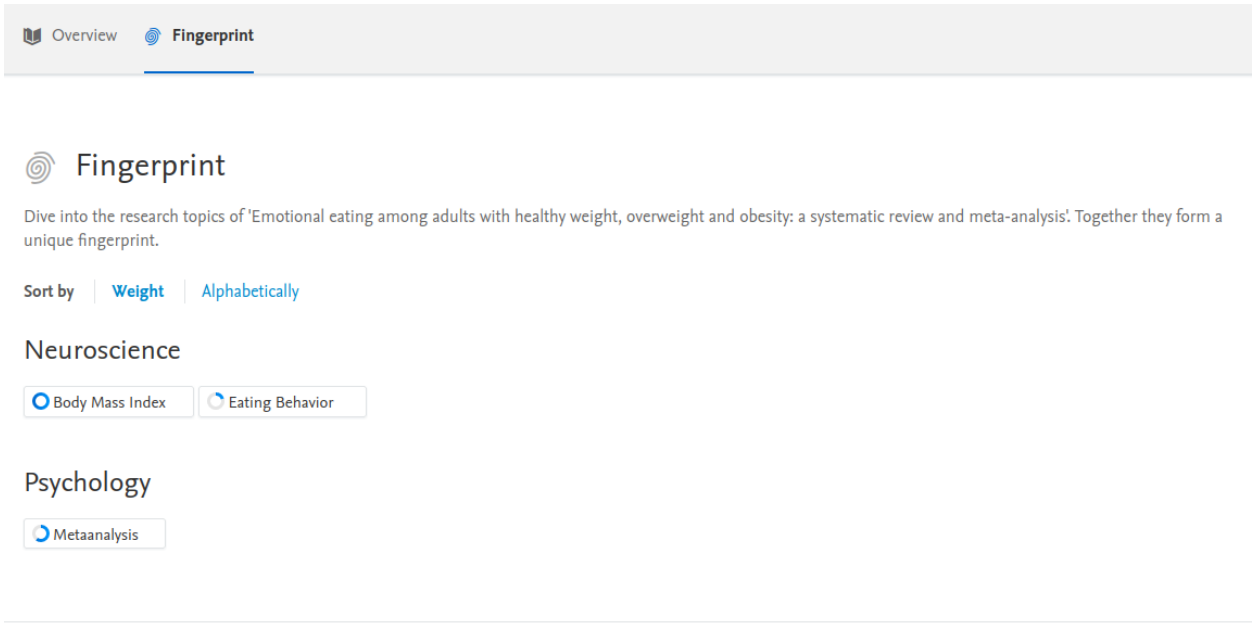


Fig 12: Crawler redirects and finds fingerprint from a publication

Schedule

Information retrieval often involves frequent fetching data from external sources, such as websites or APIs. By using a scheduler, it can be ensured that the retrieval task runs at specific intervals, ensuring that the most up-to-date information is available.

```
import schedule
import time

schedule.every().monday.at("12:00").do(
    lambda: initCrawlerScraper("https://pureportal.coventry.ac.uk/en/organisations/coventry-university/persons/")
)


while True:
    schedule.run_pending()
    time.sleep(1)
```

Fig 13: Schedule to crawl every week

The code provided sets up an automated schedule to run a specific function every Monday at noon. The function is responsible for collecting information from a website. The code keeps checking if it's time to run the function, and when the time comes, it executes the function. It then waits for a short time before checking for the next scheduled task. This process continues indefinitely.

Classification

Data preparation

From the collected data, the categorical list of values are splited into a bitmap. In a bitmap, each bit represents a specific element or attribute, and its value indicates the presence or absence of that element or attribute. If the bit is set to 1, it means the element is present in the set, and if it is set to 0, it means the element is absent.

```
In [1]: import ujson
        import pandas as pd
        import numpy as np

In [4]: data = None
        with open("./res.json", "r") as file:
            data = ujson.load(file)

In [6]: df = pd.DataFrame(data)

        df["category"] = df["category"].apply(lambda x: np.NaN if len(x) == 0 else x)
        df = df.dropna(subset=["category"])

In [21]: bitmap_df = pd.get_dummies(df['category'].apply(pd.Series).stack()).sum(level=0)

/tmp/ipykernel_280511/734249443.py:1: FutureWarning: Using the level keyword in DataFrame and Series aggregations i
s deprecated and will be removed in a future version. Use groupby instead. df.sum(level=1) should use df.groupby(le
vel=1).sum().
  bitmap_df = pd.get_dummies(df['category'].apply(pd.Series).stack()).sum(level=0)

In [23]: result_df = pd.concat([df, bitmap_df], axis=1)

In [27]: result_df.drop(columns=["category"])

Out[27]:
```

	name	pub_url	cu_author	date	abstract	Agricultural and Biological Sciences	Arts and Humanities	Biochemistry, Genetics and Molecular Biology	Chemical Engineering	Chemistry	...	Material Science
0	INHALE pause Exhale	https://pureportal.coventry.ac.uk/en/publicati...	Louise Adkins	1 Mar 2023	INHALE pause Exhale is a series of large-scale...	0	1	0	0	0	...	0
1	A shared boundary object: Financial innovation...	https://pureportal.coventry.ac.uk/en/publicati...	Samir Alamad	May 2021	This paper explores the way in which ethico-fa...	0	1	0	0	0	...	0
2	Corporate Political Activity and	https://pureportal.coventry.ac.uk/en/publicati...	Daniel Acheaw	Jan 2023	There is significant research on the	0	0	0	0	0	...	0

Fig 14: Data preparation and bitmap for classifier

Train Test Split

The DataFrame is splited into train_data and test_data, with a test size of 20% of the total data. The random_state parameter is set to 42 to ensure reproducibility of the split.

```
In [36]: train_data, test_data = train_test_split(result_df, test_size=0.2, random_state=42)

In [39]: X_train=removeStopWord(train_data["abstract"])
         X_test=removeStopWord(test_data["abstract"])

In [41]: categories = ['Computer Science',
                       'Medicine and Dentistry',
                       'Veterinary Science and Veterinary Medicine',
                       'Engineering']
         y_train = train_data[categories]
         y_test = test_data[categories]
```

Fig 15: Data splitting

The categories of interest for classification are defined as a list called categories. These categories represent the target variables. The corresponding category labels for the training and testing sets are extracted from the "train_data" and "test_data", respectively, using the specified category names. The category labels are stored in y_train and y_test.

Model Selection

The pipeline consists of two main steps: tfidf and nb:

```
In [42]: # Create a pipeline
         pipeline = Pipeline([
             ('tfidf', TfidfVectorizer()),
             ('nb', ClassifierChain(MultinomialNB()))
         ])

In [43]: # Train the classifier
         pipeline.fit(X_train, y_train)

Out[43]: Pipeline
           TfidfVectorizer
           nb: ClassifierChain
           base_estimator: MultinomialNB
             MultinomialNB

In [44]: # predict
         predictions = pipeline.predict(X_test)
```

Fig 16: Model pipelining

The TfidfVectorizer transforms the raw text data into a numerical representation called TF-IDF (Term Frequency-Inverse Document Frequency). It calculates the importance of each word in a document relative to the entire corpus. This step helps to capture the significance of words in distinguishing between different documents.

The next step represents the classifier used in the pipeline, which is a ClassifierChain wrapped around a MultinomialNB (Multinomial Naive Bayes) model. The ClassifierChain is a multi-label classifier that builds a chain of binary classifiers to predict multiple labels simultaneously. MultinomialNB is a probabilistic classifier suitable for text classification tasks.

Model Evaluation

The following calculates the accuracy of the model by comparing the predicted labels (predictions) with the true labels (y_test). The accuracy score indicates the proportion of correctly predicted labels out of the total number of labels. The F1 Scrore is a measure of the model's accuracy that considers both precision and recall. It is computed for each category (micro-average) and provides an overall measure of the model's performance. The classification_report function generates a comprehensive report that includes precision, recall, and F1-score for each category, as well as support (the number of samples) for each category. It also provides average metrics such as macro average, weighted average, and samples average.

```
In [69]: print('Accuracy = ', accuracy_score(y_test,predictions))
         print('F1 score is ',f1_score(y_test, predictions, average="micro"))
         print(classification_report(y_test,predictions))
```

```
Accuracy =  0.5661061655126937
F1 score is  0.26201315123925134
              precision    recall  f1-score   support

           0       0.99      0.20      0.34       832
           1       1.00      0.04      0.07       378
           2       0.00      0.00      0.00         7
           3       0.85      0.16      0.26       486

   micro avg       0.95      0.15      0.26      1703
   macro avg       0.71      0.10      0.17      1703
weighted avg       0.95      0.15      0.26      1703
 samples avg       0.09      0.08      0.08      1703
```

Fig 17: Model accuracy score

In the given example, the accuracy of the model is 0.5661, indicating that 56.61% of the labels were predicted correctly. The F1 score is 0.2620, which represents a trade-off between precision and recall. The macro average, weighted average, and samples average provide aggregated metrics across all categories.

The confusion matrix describes the performance of a classification model by displaying the counts of true positive, true negative, false positive, and false negative predictions.

```
In [70]: #Confusion Matrix
mat = confusion_matrix(y_test.values.argmax(axis=1), predictions.argmax(axis=1))
sns.heatmap(mat.T, square=True, annot=True, fmt='d',  cbar=False)
plt.xlabel('true label')
plt.ylabel('predicted label')
plt.show()
```

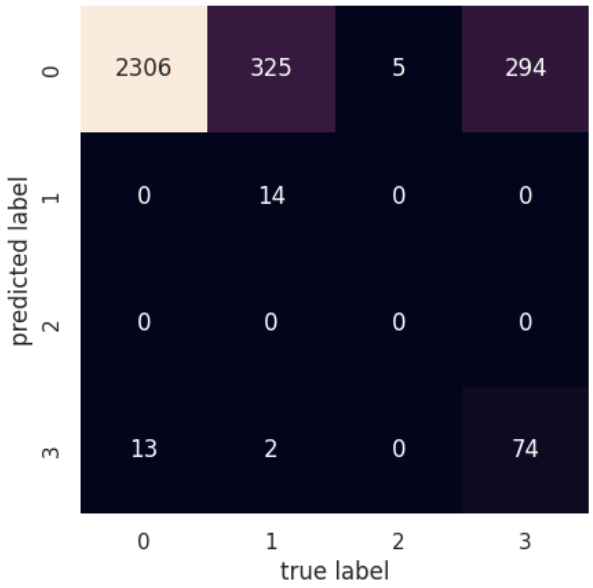


Fig 18: Model confusion matrix

The first row indicates the predictions for the first class. It shows that there are 2306 true positive predictions (correctly predicted as the first class), no false positive predictions (incorrectly predicted as the first class), no false negative predictions (incorrectly predicted as a different class), and 13 true negative predictions (correctly predicted as a different class). The other 3 rows or labels can be interpreted the same.

Search Engine

Pre-processing

First, it converts the text to lowercase using the lower() method. This ensures that the text is treated uniformly and case-insensitive operations can be performed. Next, it uses regular expressions (re.sub()) to remove any non-alphabetic characters from the text, keeping only letters. This step helps eliminate punctuation, symbols, and other non-alphabetic characters that may not be relevant for subsequent processing. The text is then split into individual words using the split() method, creating a list of words. In the following step, a list comprehension is used to filter out any stop words from the list of words. Stop words are common words such as "a," "the," or "and," which typically do not carry significant meaning and are often removed to focus on more informative content. The stop_words variable refers to a set of stop words, which is typically predefined.

```
48  def stopWordClean(text):
49      text = text.lower()
50      text = re.sub(r"[^a-zA-Z\s]", "", text)
51      words = text.split()
52
53      words = [word for word in words if word not in stop_words]
54      stemmer = PorterStemmer()
55      lemmatizer = WordNetLemmatizer()
56      words = [lemmatizer.lemmatize(stemmer.stem(word)) for word in words]
57      processed_text = " ".join(words)
58
59      return processed_text
```

Fig 19: Query preprocessing

After filtering out stop words, the function initializes a stemmer (PorterStemmer) and a lemmatizer (WordNetLemmatizer) to perform stemming and lemmatization on the remaining words. Stemming is the process of reducing words to their base or root form (e.g., "running" becomes "run"), while lemmatization aims to convert words to their dictionary or base form (e.g., "better" becomes "good"). Both techniques help consolidate different forms of words into their common representations, which can aid in information retrieval tasks. Finally, the processed words are joined back into a single string using the join() method, with each word separated by whitespace. This creates the processed_text, which is the cleaned and preprocessed version of the original text.

Indexing

```
69
70  def makeInvertedIndex(documents):
71      list_inverted_index = defaultdict(list)
72
73      for doc_id, doc in enumerate(documents):
74          terms = doc.split()
75          for term in terms:
76              list_inverted_index[term].append(doc_id)
77
78      return list_inverted_index
79
80
81  list_inverted_index = makeInvertedIndex(publication_list_after_stem)
```

Fig 20: Inverted index implementaion

The function initializes an empty “list_inverted_index” using the defaultdict class, which automatically creates a default value (an empty list) for any missing key. This ensures that every term encountered in the documents will have an associated list in the inverted index, even if the term doesn't appear in some documents. The function then iterates over each document in the documents. For each document, the text is split into individual terms (words), assuming that terms are separated by whitespace. Next, the function iterates over each term in the document. For each term, it appends the corresponding document index (doc_id) to the list associated with that term in the list_inverted_index. If a term appears multiple times in the same document, its document index will be appended multiple times to the list. Finally, the function returns the “list_inverted_index”, which now contains the mapping of terms to the documents they appear in. This inverted index can be used for efficient information retrieval tasks, such as finding documents that contain specific terms or performing full-text search.

Vectorization

TF-IDF (Term Frequency-Inverse Document Frequency) index is a numerical representation used to measure the importance of a term in a document or a collection of documents. It considers the term's frequency within a document (TF) and its rarity across the collection (IDF). By calculating TF-IDF weights, the index helps rank documents based on their relevance to a query. Search engines commonly utilize TF-IDF indexing for information retrieval.

```
83  def search(query=''):
84      totalData = len(dict_search)
85      data = dict_search
86      if query == '':
87          return {
88              'data': data,
89              'totalData': totalData
90          }
91      processed_query = stopWordClean(query)
92      query_terms = processed_query.split()
93      vectorizer = TfidfVectorizer()
94      tfidf_matrix = vectorizer.fit_transform(publication_list_after_stem)
95      relevant_docs = set()
96      for term in query_terms:
97          if term in list_inverted_index:
98              relevant_docs.update(list_inverted_index[term])
99
100     relevant_docs = list(relevant_docs)
101     tfidf_query = vectorizer.transform([processed_query])
102     cosine_similarities = cosine_similarity(tfidf_query, tfidf_matrix[relevant_docs])
103     sorted_docs = sorted(zip(relevant_docs, cosine_similarities[0]), key=lambda x: x[1], reverse=True)
104     data = [dict_search[idx] for idx, _ in sorted_docs]
105     return {
106             'data': data,
107             'totalData': len(data)
108         }
109
```

Fig 21: TFIDF implementaion

When a user enters a query, the search engine calculates the TF-IDF scores for the query terms against the documents in its database. The documents with higher TF-IDF scores for the query terms are considered more relevant and are displayed as the top search results. For example, consider a user searching for "machine learning algorithms." The search engine calculates the TF-IDF scores for the terms "machine," "learning," and "algorithms" across its indexed documents. The documents containing these terms frequently (high TF) but appearing in fewer documents overall (high IDF) will receive higher TF-IDF scores. As a result, documents that specifically discuss machine learning algorithms will be ranked higher in the search results.

TF-IDF enables search engines to accurately determine the relevance of documents to a user's query, improving the precision and effectiveness of search results. It helps retrieve documents that contain the specific terms the user is looking for and minimizes the impact of common terms that may appear in many search results.

User Interface

Search

The search UI provides optimal input for users to enter a search query. When a text is empty it will try to show all the data in the database, if the user inputs data the search query will show the optimal search results. The database saved for the model is vectorized in a tfidf vector and the search query will be vectorized in the same way. Aftwards the cosine similarity between the vectors are measured to find the matching results.

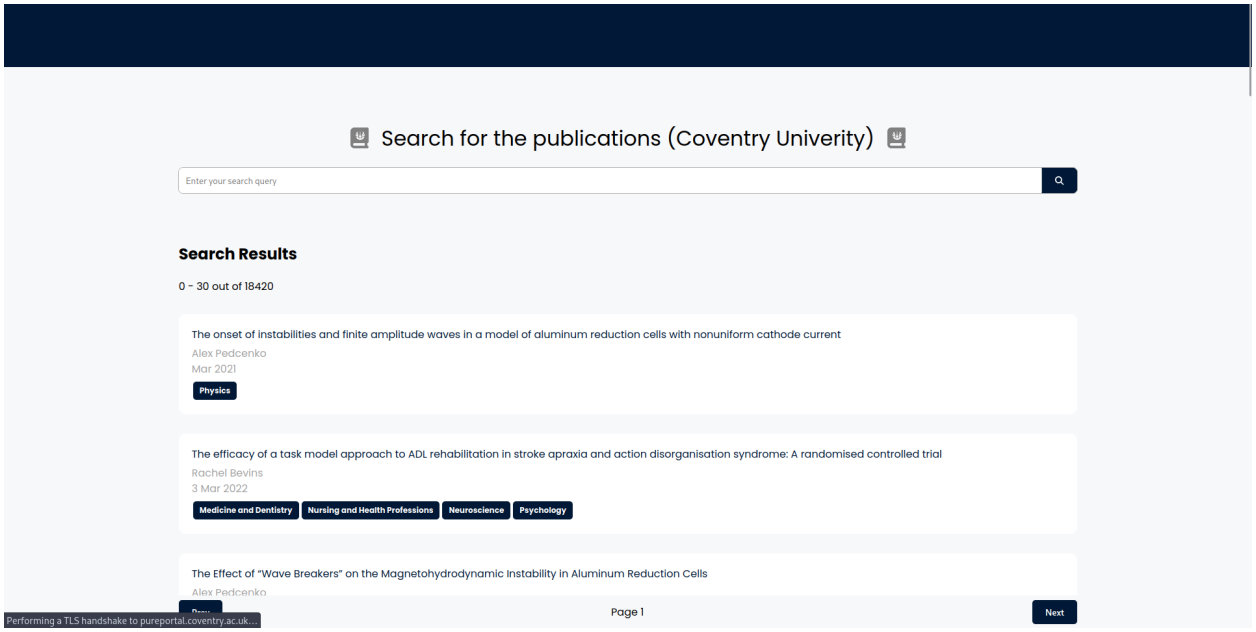


Fig 22: Search interface

The search result will be redirected to its original source if clicked as follows.

Fig 23: Search redirection to original

Comparision

The following search query is the comparison between the project's search interface and the google scholar search interface.

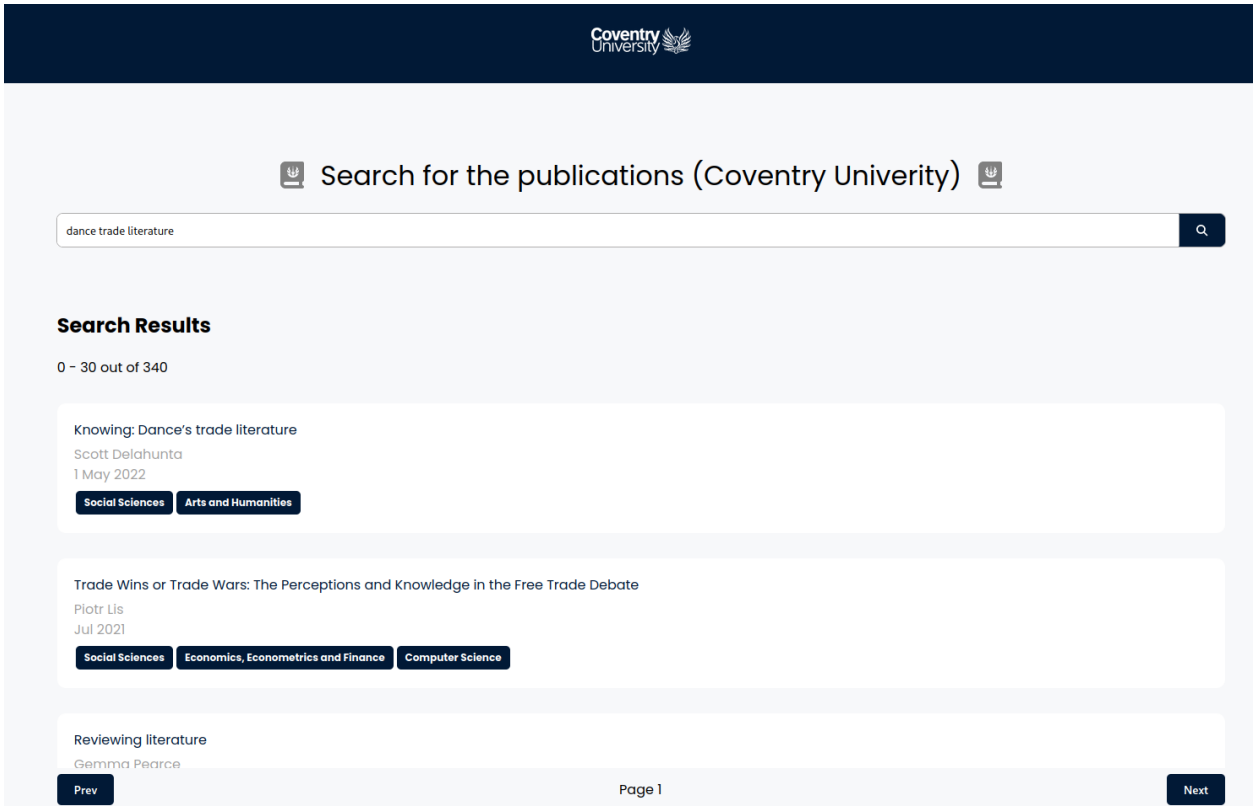


Fig 24: Search comparison (interface)

Here the top result shows the "Knowing: Dance' trade literature", the same seach query has been compared to google scholar's search engine.

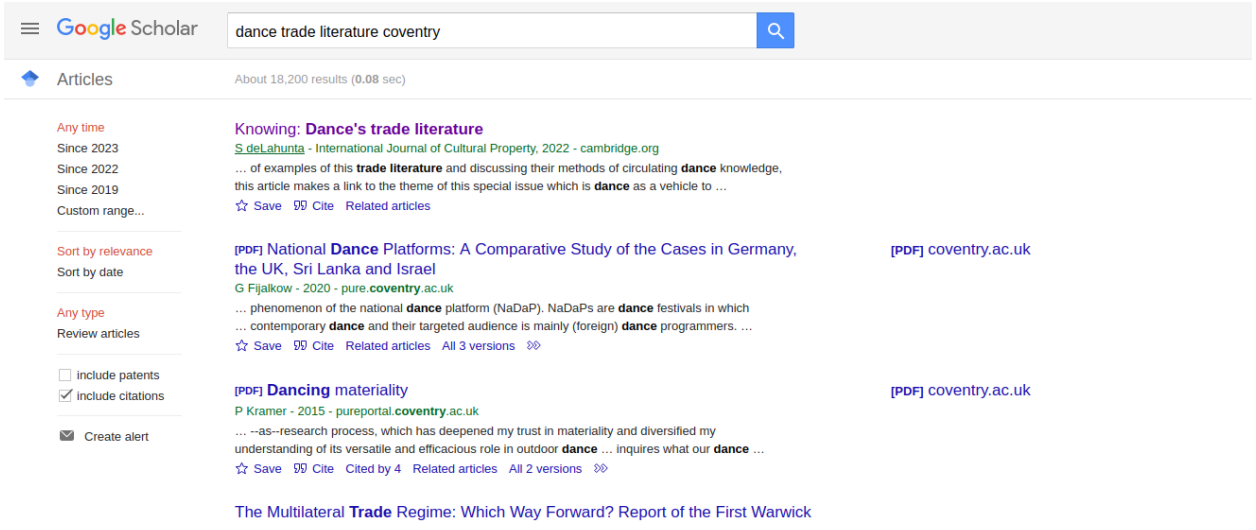


Fig 25: Search comparison (google scholar)

The result shows a similar search result, the punctuation and the case of search params has been handled in the pre-processing of the seach engine to get the optimal results.

Partial Search Query (Stemming/Lemmatizer)

The following comparison shows the advantage of using stemming and lemmantizer on the search query. The search query tested is to find the top related publication for “develop digital competence framework digital teach”.

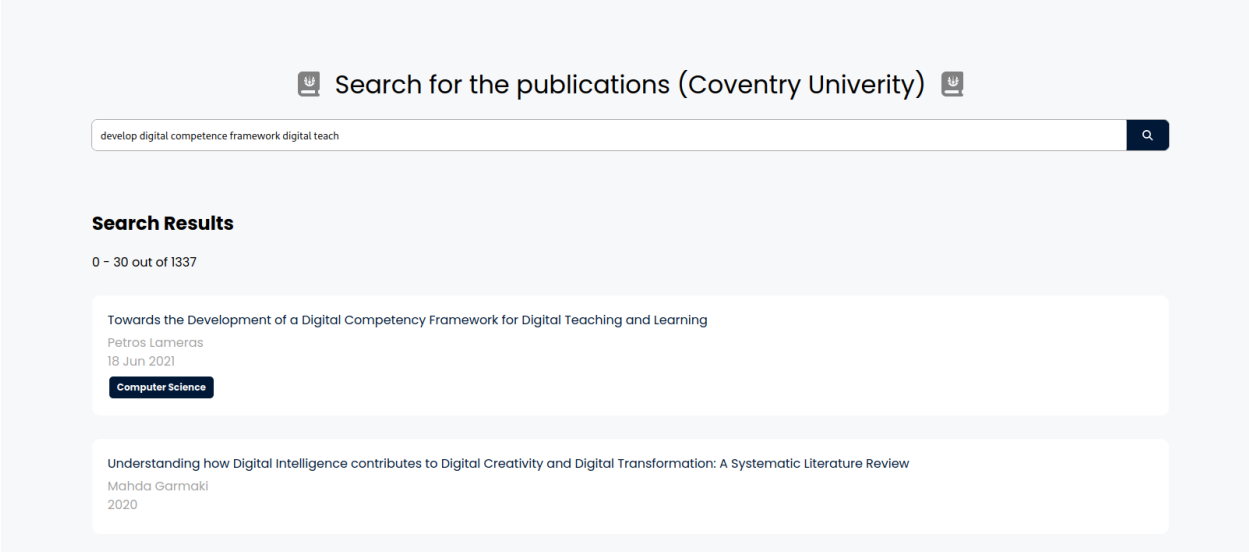


Fig 26: Search comparison (interface) 2

The stemming of the word “development”, “teaching” can be searched with the root word “develop” and “teach”. Likewise the lemmanted word for “competency” with “competence” will still be able to search the related contents.

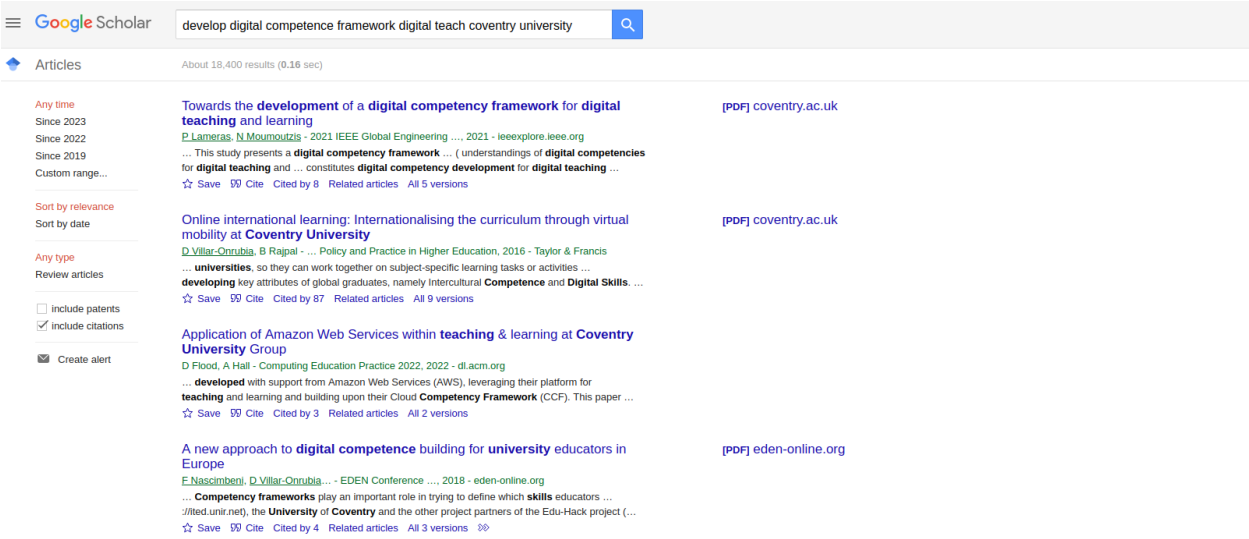


Fig 27: Search comparison (google scholar) 2

Conclusion

This project focuses on developing a vertical search engine for accessing papers and books published by Coventry University (CU) members. It employs web crawling techniques to extract data from CU academic profiles and indexes it using an inverted index structure. Users can enter queries related to their desired resources, and the system retrieves search results ranked by relevance, exclusively focusing on CU-affiliated articles. The search engine can be customized for specific disciplines. Additionally, the system includes a subject classification feature and utilizes a crawler for web data retrieval. With a scheduling mechanism in place, the system ensures timely updates for the most up-to-date information.

References

Crawler. What is a web crawler and how does it work? (n.d.). https://en.ryte.com/wiki/Crawler

Web crawlers - top 10 most popular. KeyCDN. (n.d.). https://www.keycdn.com/blog/web-crawlers

Products, D. (2022, October 4). *Text preprocessing techniques in Natural Language Processing*. Medium. https://medium.com/@dataproducts/text-preprocessing-techniques-in-natural-language-processing-19a840bb7a4f

Author: Fatih Karabiber Ph.D. in Computer Engineering, Fatih Karabiber Ph.D. in Computer Engineering, Psychometrician, E. R., & LearnDataSci, E. B. F. of. (n.d.). *TF-idf - term frequency-inverse document frequency*. Learn Data Science - Tutorials, Books, Courses, and More. https://www.learndatasci.com/glossary/tf-idf-term-frequency-inverse-document-frequency/

Multi-label classification with Scikit-MultiLearn. Section. (n.d.). https://www.section.io/engineering-education/multi-label-classification-with-scikit-multilearn/

Goyal, C. (2021, June 22). *Part 4: Step by step guide to master NLP - text cleaning techniques*. Analytics Vidhya. https://www.analyticsvidhya.com/blog/2021/06/part-4-step-by-step-guide-to-master-natural-language-processing-in-python/

F1 score in Machine Learning: Intro & Calculation. V7. (n.d.). https://www.v7labs.com/blog/f1-score-guide

Beheshti, N. (2022, February 10). *Guide to confusion matrices & classification performance metrics*. Medium. https://towardsdatascience.com/guide-to-confusion-matrices-classification-performance-metrics-a0ebfc08408e

Appendix

https://github.com/albertmaharjan94/information-retreival
https://youtu.be/_kBoB7SuTb4